

FREE GUIDE

Scope Your App Like a Founder, Not a Salesperson.

The five questions to ask before you build, what a realistic timeline looks like, and how to tell a good build partner from a bad one.

Written by

Soni Consultancy Services

React Native & MERN app development studio.
Products live on the App Store & Google Play.

soniconsultancyservices.com

het.soni@soniconsultancyservices.com

START HERE

Why most app projects go wrong before a line of code

Most failed app builds are not failures of engineering. They are failures of scoping — decisions made, or avoided, in the two weeks before anyone opened an editor.

Three patterns cause most of the damage:

Scope creep. Features get added mid-build because nobody decided up front what was version one and what was later. The timeline doubles and the budget follows it.

The wrong technical choice. A stack gets picked because it is familiar to someone in the room, not because it fits the product. The bill arrives six months later, as a rewrite.

The wrong build partner. The cheapest quote wins because the buyer had no way to tell a serious proposal from an optimistic one. What ships cannot scale, and the rescue costs more than doing it properly would have.

None of these need deep technical knowledge to avoid. They need better questions, asked earlier. That is what this guide is.

This guide is deliberately vendor-neutral. Whether you build with us, hire in-house, or go to another studio entirely, the framework below is the same.

SECTION ONE

The five questions to answer before you build

Answer these honestly and you have a real scope. Skip them and you have a wish list that will be priced as one.

1. What is the one thing this product must do to be worth launching?

Not the five things. The one. If you cannot finish the sentence "this is a failure unless it can ____", the scope is not ready. Everything that does not serve that sentence is version two.

Good sign: you can state it in a sentence, without "and".

Warning sign: every feature feels equally essential.

2. Who is the first user, and what do they do today instead?

Every product replaces something — a spreadsheet, a phone call, a competitor, or doing nothing at all. The thing you are replacing sets the bar you have to clear, and it tells you which features are genuinely required on day one.

Good sign: you can name a specific person and describe their current workaround.

Warning sign: the answer is a demographic, not a person.

3. What has to be true on the back end?

The screens are the visible part and usually the cheaper part. Real-time updates, payments, offline behaviour, role-based access, third-party integrations and data volume drive far more of the cost than screen count does. This is the question that most often moves a quote by a factor of two.

Good sign: you have listed the systems it must talk to.

Warning sign: "it's just a simple app".

4. What is the real constraint — the date, the budget, or the scope?

One of these three is fixed and the other two have to flex. If you do not decide which, the decision gets made for you halfway through, usually badly. A team that knows your real constraint can protect it. A team that doesn't will optimise for the wrong one.

Good sign: you can say which of the three you would sacrifice first.

Warning sign: all three are non-negotiable.

5. What happens the day after launch?

Launch is the start of the cost, not the end of it. App stores require ongoing releases, OS versions change, and real usage produces work that no plan anticipated. Budgets that stop at launch day tend to strand the product a few months later.

Good sign: there is an owner and a budget line for month four.

Warning sign: the plan ends at "live on the store".

SECTION TWO

Cutting scope to a real version one

The fastest way to a shippable scope is to sort every feature into three buckets and be strict about the first one.

Core. Without it, the product does not do its one job. If you can describe a usable product without the feature, it is not core.

Supporting. It makes the core work properly — sign-in, basic settings, whatever the core genuinely depends on. Keep this list short and suspicious.

Later. Everything else. Not "cut" — scheduled. This distinction matters, because a feature that is

written down as later stops being argued about weekly.

A practical test: for each feature, ask what actually happens if it ships three months after launch instead. If the answer is "some users would find it inconvenient", it is not core. If the answer is "the product makes no sense", it is.

SECTION THREE

What a realistic timeline looks like

Timelines below are **engineering effort** — the working time of a senior team — not calendar time with more people added. Adding people to a late project reliably makes it later, because coordination cost grows faster than output.

These bands come from products we have actually shipped to the App Store and Google Play. Client names are withheld under NDA; the durations are real.

MVP / simple — roughly 10 to 16 weeks

A focused product proving one core loop: a handful of screens, standard authentication, a straightforward backend, one or two integrations. This is what you build to test demand. Typically one to two senior engineers with part-time design. An attendance and payroll platform we built shipped in around 16 weeks, at the fuller end of this band.

Standard / market-ready — roughly 24 to 32 weeks

A polished product with several connected modules, custom interface work, real-time features, payments and a proper admin side. Most funded products land here. Typically two to four engineers plus dedicated design and QA. A creator marketplace (24 weeks) and a retail operations platform (32 weeks) we built both sit in this range.

Complex / scale — 32 weeks and up

Multi-sided platforms, heavy integrations, AI features, strict compliance requirements or high concurrency. Cost here scales with the integration and reliability surface, not with screen count. An AI ride-hailing platform we built, with live tracking and fare prediction, took 25 weeks; a healthcare staffing platform took 26.

Note that the ride-hailing and healthcare examples took fewer weeks than the retail platform in the tier above. Complexity is not the same as duration — a tightly scoped complex product can ship faster than a sprawling simple one. Which is the entire argument for scoping properly.

SECTION FOUR

Choosing the stack without the folklore

"Wrong technical choice" was one of the three failure modes at the start of this guide, so it deserves a straight answer. For mobile, the realistic options are React Native, Flutter, or fully native (Swift and Kotlin, written twice).

Performance is no longer the deciding factor. React Native's New Architecture and Flutter's Impeller renderer have converged on "fast enough that users cannot tell" for mainstream products — content, commerce, social, SaaS, booking, internal tools. The performance problems that actually reach users are unoptimised lists, oversized images and chatty APIs. Those are architecture problems, and they bite equally in all three.

What actually decides it:

Who you can hire and keep. The most underweighted factor, and in our experience it decides more projects than every technical comparison combined. JavaScript and React are the largest talent pool in software, and web engineers become productive in React Native quickly. Dart is a smaller, separate hiring market — fine for a company committed to Flutter, friction for a team that needs to staff flexibly over five years.

How your UI should feel. Flutter draws every pixel itself, so your design system renders identically on every device — the right call when brand-exact rendering is the point. React Native composes real platform components, so the app feels native on each OS by default. Pick the one that matches what your product is actually trying to be.

Your five hairiest integrations. List them — the payment provider, the analytics suite, the industry-specific hardware — then search each ecosystem for maintained packages before you decide. Twenty minutes of checking beats any opinion piece, this one included.

When to go fully native. If the product's success genuinely depends on sustained heavy custom animation, deep platform integration, or a benchmark margin, write native. Otherwise you are paying for two codebases, two teams and two release trains to solve a problem you do not have.

One caveat worth budgeting for: cross-platform reduces native code, it does not abolish it. Expect some Swift or Kotlin work in any of these paths. For transparency — we build in React Native, largely for the hiring and maintenance reasons above. That is a preference, not a universal answer, and the framework above is how you should test it.

SECTION FIVE

How to think about budget

Any guide that hands you a single number for "an app" is guessing. Cost follows the engineering effort in the tiers above, and effort follows the answers to the five questions in section one — particularly question three.

As a starting frame: a focused MVP typically begins in the low five figures (USD), and full products scale from there with feature depth. That is a starting point for a conversation, not a quote.

The costs founders most often forget:

- App store developer accounts and the submission cycle
- Third-party services that bill per user or per transaction
- Hosting and infrastructure, which grow with usage
- Maintenance and OS-version updates after launch
- The second round of design once real users arrive

For a tailored range against your own scope, the calculator on our site takes about two minutes and is more useful than any blanket figure.

soniconsultancyservices.com/app-cost-calculator

SECTION SIX

How we scope a project

This is our own process, included so you know what a scoping conversation should feel like — and so you can hold any team, including us, to it.

1. Scope call — 30 minutes

Your goals, users and constraints, with an honest read on feasibility. If the idea has a serious problem, this is where you should hear it, not three months in.

2. Fixed-price proposal — within 48 hours

Scope, timeline and a fixed price. No vague day-rates that leave the final number open.

3. Build — weekly demos

Working software you can hold in your hand from the first sprint, not a status report describing it.

4. Launch and support

Store submission handled, then monitoring and iteration once real usage starts.

SECTION SEVEN

Judging a proposal

Once you have quotes in hand, these are the signals that matter more than the number at the bottom.

Worth trusting:

- They pushed back on part of your scope, and explained why
- The proposal separates version one from later phases explicitly
- Assumptions are written down, so you can see what the price depends on

- Post-launch support is priced, not left vague
- You will have direct access to the people writing the code

Worth worrying about:

- Every feature you mentioned was agreed to without question
- The estimate arrived without anyone asking about your back end
- The price is dramatically below every other quote you have
- You cannot tell who is actually going to build it
- "We'll figure that out during development" appears more than once

A team that tells you your scope is too big for your budget is doing you a favour. It is cheaper to hear it now than in month four.

CHECKLIST**Before you send a single enquiry**

You are ready to talk to any build team when you can answer these without hedging:

- The one thing the product must do to be worth launching
- Who the first user is, and what they do instead today
- What has to be true on the back end — systems, data, real-time needs
- Which of date, budget or scope is genuinely fixed
- Who owns the product, and the budget, after launch
- Your Core / Supporting / Later split, written down
- Which tier your product realistically sits in

If several of these are still blank, that is useful information — it tells you exactly what to work out before you start paying for engineering time.

NEXT STEP

If you'd like a second opinion.

A 30-minute scoping call. No obligation, no pitch deck:
your idea, an honest read on feasibility, and a fixed-price
proposal within 48 hours if it's a fit.

If it isn't a fit, we'll tell you on the call.

Book a call

soniconsultancyservices.com/contact

Estimate your scope

soniconsultancyservices.com/app-cost-calculator

Email

het.soni@soniconsultancyservices.com

Soni Consultancy Services

React Native & MERN app development, with AI where it earns its place.
Products live on the App Store & Google Play.

This guide is free to share. Please don't repackage it as your own.